ChatGPT

Hackathon-grade RWA Yield Guardian on Mantle

Executive summary

For a hackathon build, the cleanest and highest-confidence design is a **mainnet-first address model with a Sepolia harness**, not a testnet-first RWA deployment model. The strongest verified items are: Mantle mainnet/Sepolia network details from Mantle's own developer docs; Mantle mainnet USDY, mUSD, and the Mantle Redemption Price Oracle from Ondo's contract registry; the Mantle L2 mETH token from the mETH contracts repository; Merchant Moe router contracts from Merchant Moe docs; Agni mainnet router details from Agni's SDK repository; and the Mantle deployment of the Pyth pull-oracle contract from Pyth's official contract-address page. [1]

The practical implication is straightforward. On **Mantle mainnet**, you can confidently wire a guardian around USDY and mETH. On **Mantle Sepolia**, the official network is live and usable, but the gathered official sources did **not** expose current Sepolia deployments for USDY, mETH, Merchant Moe, or Agni. Those should therefore be treated as **UNSPECIFIED** and replaced in testing by mocks or a mainnet fork unless you independently discover and verify current Sepolia deployments. Mantle's current official testnet is **Mantle Sepolia, chain ID 5003**; the older 2023 Mantle testnet material references chain ID 5001 and should not be used as the current source of truth. [2]

For pricing, the most robust pattern is to use **Pyth Network** [3] **for ETH/USD** and to use **Ondo Finance** [4] **'s Mantle Redemption Price Oracle as the primary USDY reference**, because Ondo explicitly documents that oracle on Mantle and states that it stores the current and historical redemption price. In the material gathered for this report, no official Pyth USDY feed was confirmed; the correct fallback is therefore **Ondo oracle price + DEX market price + liquidity checks**, not "inventing" a USDY oracle. [5]

The safest architecture is an **off-chain risk engine** that computes a signed decision packet and an **on-chain executor** that only performs swaps or rebalances when all hard guards pass: fresh oracle data, acceptable deviation from Ondo oracle, adequate liquidity, non-expired risk approval, and a global pause/veto state. That pattern matches the risk surfaces explicitly documented by Ondo, Mantle, and the mETH Protocol [6] terms: legal/compliance structure and blocklist controls for USDY, liquidity-buffer/slashing/oracle dependencies for mETH, and Mantle's own network-level failure controls such as data availability, fraud proofs, and forced transaction inclusion. [7]

Assumptions and verified infrastructure

Explicit assumptions

This report assumes the production target is **Mantle mainnet** and the non-production harness is **Mantle Sepolia**. Where a testnet contract is not publicly documented in the gathered official sources, it is marked **UNSPECIFIED** rather than guessed. It also assumes the guardian is **execution-light on-chain** and **decision-heavy off-chain**, because that is the lowest-risk hackathon architecture when some RWA/reference feeds are only partially documented across networks. [8]

Network and access details

Item	Mantle mainnet	Mantle Sepolia
Chain ID	`5000`	`5003`
RPC URL	`https://rpc.mantle.xyz`	`https://rpc.sepolia.mantle.xyz`
WebSocket	`wss://rpc.mantle.xyz`	`N/A`
Explorer	`https://mantlescan.xyz/`	`https://sepolia.mantlescan.xyz/`
Faucet	`N/A`	`https://faucet.sepolia.mantle.xyz/`
Bridge	`https://app.mantle.xyz/bridge`	`https://app.mantle.xyz/bridge?network=sepolia`
Wrapped MNT	`0x78c1b0C915c4FAA5FffA6CAbf0219DA63d7f4cb8`	`0x19f5557E23e9914A18239990f6C70D68FD`

Mantle's current docs also note that the official public RPC is rate-limited and explicitly recommend third-party RPCs if your use case calls Mantle APIs frequently. For a yield guardian that does frequent quoting, event backfilling, and mempool-safe decision loops, that means you should keep the official RPC as a fallback and run a dedicated RPC provider in parallel. [9]

Address matrix

Asset / contract	Mantle mainnet	Mantle Sepolia
USDY	`0x5bE26527e817998A7206475496fDE1E68957c5A6`	**UNSPECIFIED**
mUSD	`0xab575258d37EaA5C8956EfABe71F4eE8F6397cF3`	**UNSPECIFIED**
Ondo Redemption Price Oracle	`0xA96abbe61AfEdEB0D14a20440Ae7100D9aB4882f`	**UNSPECIFIED**
Ondo Blocklist	`0xdBd7a7d8807f0C98c9A58f7732f2799c8587e5c6`	**UNSPECIFIED**

Asset / contract	Mantle mainnet	Mantle Sepolia
mETH L2 token	`0xcDA86A272531e8640cD7F1a92c01839911B90bb0`	**UNSPECIFIED**
WMNT	`0x78c1b0C915c4FAA5FffA6CAbf0219DA63d7f4cb8`	`0x19f5557E23e9914A18239990f6C70D68F`
ETH on Mantle L2	`0xdEAddEaDdeadDEadDEADDEAddEADDEAddead1111`	**UNSPECIFIED**
WETH	**UNSPECIFIED**	**UNSPECIFIED**

Asset / contract	Mantle mainnet	Mantle Sepolia
USDC	`0x09Bc4E0D864854c6aFB6eB9A9cdF58aC190D0dF9`	**UNSPECIFIED**
Merchant Moe Router	`0xeaEE7EE68874218c3558b40063c42B82D3E7232a`	**UNSPECIFIED**
Merchant Moe LB Router	`0x013e138EF6008ae5FDFDE29700e3f2Bc61d21E3a`	**UNSPECIFIED**
Merchant Moe LB Factory	`0xa6630671775c4EA2743840F9A5016dCf2A104054`	**UNSPECIFIED**
Agni SwapRouter	`0x319B69888b0d11cEC22caA5034e25FfFBDc88421`	**UNSPECIFIED**
Agni Factory	`0x25780dc8Fc3cfBD75F33bFDAB65e969b603b2035`	**UNSPECIFIED**
Agni QuoterV2	`0xc4aaDc921E1cdb66c5300Bc158a313292923C0cb`	**UNSPECIFIED**

A crucial nuance: Agni's SDK still exposes a generic `TESTNET_ADDRESSES` block, but that block's `WMNT` does **not** match Mantle's current official Sepolia WMNT. That mismatch is strong evidence that the SDK's "TestNet" set should **not** be assumed to be current Mantle Sepolia. For the current hackathon, treat Agni Sepolia addresses as **UNSPECIFIED** until verified on Sepolia explorer or by ABI checks over the Sepolia RPC. 16

How to discover anything marked UNSPECIFIED

Use the following exact checks before coding against any unresolved contract:

```
# 1) Confirm the address is a deployed contract
curl -X POST https://rpc.sepolia.mantle.xyz \
  -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_getCode","params":
["0xCANDIDATE","latest"]}'

# 2) Read symbol()
curl -X POST https://rpc.sepolia.mantle.xyz \
  -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_call","params":
[{"to":"0xCANDIDATE","data":"0x95d89b41"},"latest"]}'

# 3) Read decimals()
curl -X POST https://rpc.sepolia.mantle.xyz \
  -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_call","params":
[{"to":"0xCANDIDATE","data":"0x313ce567"},"latest"]}'
```

For DEX routers and pool factories, search the current contract labels on Sepolia explorer, then verify runtime calls such as `factory()`, `WETH9()` or equivalent router methods against the verified ABI. Where no official subgraph is documented, prefer direct `eth_getLogs` over guessing an indexer. This is the correct operational discipline for an RWA project where silent address mistakes are worse than missing an integration deadline.

Oracle and API stack

The cleanest oracle stack is:

1. **ETH/USD** from Pyth on Mantle.
2. **USDY reference price** from Ondo's Mantle Redemption Price Oracle.
3. **Market sanity** from DEX quotes/TWAPs and liquidity depth checks.
4. **mETH reference** from market quotes against ETH plus DEX liquidity and discount monitoring, because the gathered sources did not expose a separate official mETH oracle on Mantle. [17]

Oracle and API details

Component	What to use	Account / key	Permissions / auth posture	Why it matters
Mantle public RPC	Official Mantle RPCs	None	Public, but rate-limited	Fine for setup and fallback; not ideal as sole production source. [9]

Component	What to use	Account / key	Permissions / auth posture	Why it matters
Dedicated RPC	Alchemy [18] or QuickNode [19]	RPC key / endpoint token	Alchemy supports API keys, headers, JWTs, and allowlists; QuickNode uses endpoint auth token in URL or `x-token`, plus IP whitelist and method filters via admin APIs. [20]	
Pyth off-chain updates	Hermes public instance for development	Usually none for public development instance	Pyth states Hermes is available via a public development instance; provider-specific access may differ if you use third-party hosted Hermes. [21]	
Pyth on-chain contract	Mantle Pyth contract	None beyond wallet gas	On-chain `updatePriceFeeds` requires `msg.value`; Pyth lists Mantle update fees as `0.01 MNT` per feed update. [22]	
Ondo price reference	Mantle Redemption Price Oracle	None beyond RPC access	Official on-chain reference for current and historical USDY redemption price. [10]	
Explorer APIs	Mantlescan [23], which points developers to Etherscan [24] API docs/ plans	API key	Mantlescan links directly to Etherscan API Documentation and API Plans; Etherscan V2 uses a single account/API-key model across supported chains. [25]	
LLM option	Google [26] Gemini API	API key	Google's Gemini API uses `x-goog-api-key`; AI Studio can create the key directly. For this hackathon path, API-key mode is the simplest confirmed auth mode. [27]	
Local LLM option	Ollama [28]	None locally; `OLLAMA_API_KEY` for cloud	Local API needs no auth on `localhost`; direct cloud API access requires an API key. [29]	

Pyth contract and feed status

Item	Value	Status
Pyth contract on Mantle mainnet	`0xA2aa501b19aff244D90cc15a4Cf739D2725B5729`	Officially documented. [30]
ETH/USD feed ID	`0xff61491a931112ddf1bd8147cd1b641375f79f5825126d665480874634fd0ace`	Officially surfaced in Hermes docs as the ETH/USD example. [21]
USDY feed ID	**UNSPECIFIED**	No official USDY Pyth feed was confirmed in the gathered sources. Use Ondo oracle instead.
Equivalent stable / RWA sanity feed	USDC/USD from Pyth	**Partially confirmed only**: official snippet confirms a USDC/USD feed exists and begins `eaa020...` / ends `...c94a`, but the full ID was not captured in the gathered material. Treat the full hex as **UNSPECIFIED** until copied from the official Pyth feed list. [31]

Item	Value	Status
Pyth Mantle Sepolia contract	**UNSPECIFIED**	Not captured in the gathered official material. Discovery required.

Recommended integration sequence

Step	Action	Notes
Resolve network config	Hardcode chain IDs 5000/5003, official RPCs, explorers, and WMNT addresses in an immutable config layer	These are the most stable constants in the stack. [9]
Resolve mainnet contract registry	Store USDY, mUSD, Ondo Redemption Price Oracle, mETHL2, Merchant Moe routers, Agni router, and Pyth contract in versioned config	Separate “verified” and “candidate” addresses. [32]
Build oracle adapters	`PythEthUsdAdapter`, `OndoUsdyOracleAdapter`, `DexQuoteAdapter`	Never use DEX spot alone for RWA decisions. [33]
Build liquidity adapters	Quote both Merchant Moe LB Router and Agni SwapRouter; compare output and effective depth	For Merchant Moe, prefer LB Router when targeting LB pools. [34]
Build hard-risk engine	Compute score plus boolean vetoes; output signed decision packet	Keep heuristics off-chain, enforcement on-chain.
Build executor	On-chain checks for pause, signer approval, expiry, max slippage, oracle freshness, and risk-score threshold	This is the core guardian behaviour.
Build ops hooks	Store every decision, emitted event, and alert to Telegram/Discord/webhook	Hackathon judges like explainability and traceability.
Test realistic behaviour	Use mainnet fork for USDY/mETH realism; use Mantle Sepolia for pause/unpause and signature flow	Because gathered sources do not confirm these RWA assets on Sepolia. [35]

Risk management framework

The most defensible risk policy for this project is to treat **USDY** and **mETH** as two different risk families.

For **USDY**, the dominant risks are: issuer/legal/compliance structure, blocklist and transfer restrictions, deviation of market price from redemption/oracle price, liquidity depth in the on-chain venue, and operational risk around pricing/oracle freshness. Ondo documents USDY as a special-purpose vehicle

issuing debt obligations backed by US Treasuries and US bank demand deposits, offered under Regulation S to non-US persons, with AML/sanctions controls and a blocklist contract. [36]

For **mETH**, the key risks are different: slashing, socialised exposure, liquidity-buffer dependence, third-party protocol risk, oracle risk in those external protocols, governance/parameter changes in external protocols, and slow redemptions under stress. Those risks are all spelled out in the published mETH terms, including interaction with third-party protocols such as Aave-like markets and the explicit statement that accelerated redemption is not guaranteed. [37]

At the chain level, Mantle itself identifies data availability, fraud proofs, and forced transaction inclusion as core risk-management mechanisms. That argues for one more top-level rule in your guardian: **if chain-health assumptions are degraded, stop execution and only monitor**. [38]

Recommended hard rules

The following table is a **design recommendation inferred from the documented risk surfaces above**, not a claim that any protocol mandates these exact thresholds. It is intended to be hackathon-grade, readable, and defendable. [39]

Control	Measurement	Warn	Veto
USDY depeg vs Ondo oracle	`abs(dex_mid / ondo_redemption_price - 1)`	`> 0.0075`	`> 0.0150`
mETH discount vs ETH	`1 - (mETH_mid / ETH_mid)`	`> 0.0100`	`> 0.0250`
Pyth confidence ratio	`conf / abs(price)`	`> 0.0030`	`> 0.0075`
Oracle staleness	seconds since last valid oracle update	`> 180`	`> 600`
Instant liquidity coverage	`depth within 1% band / intended_trade_notional`	`< 5x`	`< 2x`
Execution slippage	expected route slippage	`> 0.50%`	`> 1.00%`
Portfolio concentration in one RWA	`asset_exposure / NAV`	`> 35%`	`> 50%`
Counterparty concentration	issuer/custodian-weighted exposure	`> 40%`	`> 60%`
Yield instability	7d annualised yield delta	`> 100 bps`	`> 250 bps` or negative net carry
Upgrade / governance shock	proxy upgrade, privileged role change, or external protocol parameter change very recently	manual review	automatic pause for 24h
Chain-health degradation	sequencer/explorer/oracle outage or inconsistent quoting	manual review	automatic pause

Quantitative risk score

A useful composite score is:

$$\text{RiskScore} = 100 \times (0.25M + 0.20L + 0.15Y + 0.15C + 0.10S + 0.10O + 0.05X)$$

Where:

- M = market dislocation subscore
- L = liquidity subscore
- Y = yield instability subscore
- C = counterparty / custodial / issuer subscore
- S = smart-contract / upgrade subscore
- O = oracle quality subscore
- X = execution / chain-health subscore

Each subscore is normalised onto $[0, 1]$. As a governance rule:

- **0–29**: allow
- **30–49**: allow but shrink size by 50%
- **50–69**: no new allocations, only reduce risk
- **70–100**: veto and pause strategy for that asset

This weighting is deliberate. It prioritises **market dislocation and liquidity** for hackathon execution safety, while still keeping meaningful weight on counterparty/legal structure for RWAs and third-party protocol/oracle dependencies for mETH. That weighting is an inference from Ondo's documented SPV/oracle/blocklist model, mETH's documented slashing/liquidity-buffer dependencies, and Mantle's network-level failure modes. [40]

Example calculations

These are illustrative calculations for policy tuning, not live market observations.

Scenario	M	L	Y	C	S	O	X	Score	Outcome
USDY in normal conditions	0.12	0.15	0.10	0.35	0.20	0.10	0.10	16.8	Allow
USDY with 1.2% market/oracle gap and thin pool	0.65	0.55	0.15	0.35	0.20	0.15	0.20	40.0	Shrink / caution
mETH with 2.8% discount and stressed exits	0.85	0.80	0.35	0.25	0.45	0.20	0.35	58.0	Reduce only
Any asset with stale oracle and chain-health issue	0.40	0.40	0.20	0.25	0.25	1.00	1.00	47.8 + hard veto	Pause

Reference architecture and code

Architecture

```
flowchart LR
    U[Strategy scheduler] --> D[Data adapters]
    D --> R1[Mantle RPC]
    D --> R2[Pyth Hermes]
    D --> R3[Ondo Redemption Oracle]
    D --> R4[DEX quote adapters]
    D --> R5[Explorer / logs]

    D --> E[Risk engine]
    E --> S[(Risk DB / event store)]
    E --> A[Alerting layer]
    E --> P[Signed decision packet]

    P --> X[On-chain executor]
    X --> Y[Pause / veto guard]
    X --> Z[DEX router call]

    Y --> Z
    Z --> MM[Merchant Moe]
    Z --> AG[Agni]
```

This split keeps volatile logic off-chain and keeps funds protection on-chain. That matches the reality of the documented surfaces: Pyth is a pull oracle whose updates are fetched from Hermes and pushed on-chain with a fee, Ondo provides an on-chain redemption oracle for USDY on Mantle, and Mantle itself emphasises safety mechanisms at the network level rather than promising every external integration will always be healthy. [41]

Decision flow

```
flowchart TD
    A[Fetch ETH/USD from Pyth] --> B[Fetch USDY redemption price from Ondo]
    B --> C[Fetch DEX quotes and depth]
    C --> D[Compute risk score]
    D --> E{Any hard veto?}

    E -- yes --> F[Emit veto event]
    F --> G[Pause asset strategy]

    E -- no --> H{Score < threshold and approval fresh?}
    H -- no --> F
    H -- yes --> I[Send signed decision packet]
    I --> J[Executor updates Pyth if needed]
    J --> K[Executor re-checks slippage/oracle freshness]
```

```
    K --> L[Execute swap or rebalance]
    L --> M[Emit execution event]
```

Solidity template

The Solidity below is intentionally minimal. It shows the correct **control pattern**: signed approval, pause guard, oracle freshness check, and emitted veto events. Replace placeholder interfaces with the exact verified ABIs from the deployed contracts before production use.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.23;

interface IPythLike {
    function getUpdateFee(bytes[] calldata updateData) external view returns
(uint256);
    function updatePriceFeeds(bytes[] calldata updateData) external payable;
}

interface IOndoOracleLike {
    // Replace with the exact verified ABI from the Mantle deployment.
    function getPrice() external view returns (uint256 priceE18, uint256
updatedAt);
}

contract RwaYieldGuardianExecutor {
    address public owner;
    address public riskSigner;
    bool public paused;

    uint256 public maxAge = 600;          // seconds
    uint256 public maxRiskScore = 49;     // allow only < 50
    uint256 public maxOracleDeviationBps = 150; // 1.50%

    event Paused(address indexed by, bool isPaused);
    event RiskVeto(bytes32 indexed asset, uint256 score, string reason);
    event ExecutionApproved(bytes32 indexed asset, uint256 score, uint256
amountIn);
    event ExecutionBlocked(bytes32 indexed asset, string reason);

    modifier onlyOwner() {
        require(msg.sender == owner, "not owner");
        _;
    }

    modifier notPaused() {
        require(!paused, "paused");
        _;
    }

    constructor(address _riskSigner) {
```

```
        owner = msg.sender;
        riskSigner = _riskSigner;
    }

    function setPaused(bool v) external onlyOwner {
        paused = v;
        emit Paused(msg.sender, v);
    }

    struct RiskApproval {
        bytes32 asset;
        uint256 riskScore;
        uint256 expiresAt;
        uint256 oracleRefPriceE18; // Ondo or ETH ref price
        uint256 maxDeviationBps;
    }

    function executeIfSafe(
        RiskApproval calldata approval,
        bytes calldata /* signature */,
        address pyth,
        address ondoOracle,
        bytes[] calldata pythUpdateData,
        uint256 dexObservedPriceE18,
        uint256 amountIn
    ) external payable notPaused {
        if (approval.expiresAt < block.timestamp) {
            emit ExecutionBlocked(approval.asset, "approval expired");
            revert("approval expired");
        }

        if (approval.riskScore > maxRiskScore) {
            emit RiskVeto(approval.asset, approval.riskScore, "risk too
high");
            revert("risk too high");
        }

        // TODO: verify signature over approval using EIP-712.
        // Keeping omitted here to keep the template short.

        if (pythUpdateData.length > 0) {
            uint256 fee = IPythLike(pyth).getUpdateFee(pythUpdateData);
            require(msg.value >= fee, "insufficient pyth fee");
            IPythLike(pyth).updatePriceFeeds{value: fee}(pythUpdateData);
        }

        (uint256 refPriceE18, uint256 updatedAt) =
IOndoOracleLike(ondoOracle).getPrice();
        require(refPriceE18 > 0, "bad ref price");
        require(block.timestamp - updatedAt <= maxAge, "stale oracle");
```

```
        uint256 diff = dexObservedPriceE18 > refPriceE18
            ? dexObservedPriceE18 - refPriceE18
            : refPriceE18 - dexObservedPriceE18;

        uint256 deviationBps = diff * 10_000 / refPriceE18;
        require(deviationBps <= approval.maxDeviationBps, "oracle
deviation");

        emit ExecutionApproved(approval.asset, approval.riskScore, amountIn);

        // TODO: perform router call only after all checks pass.
    }
}
```

This template is appropriate because Pyth on EVM requires fetching update data from Hermes and paying an update fee on-chain, while Ondo exposes a dedicated Mantle redemption oracle for USDY. The important idea is that the executor never "decides"; it only **enforces**. [42]

Python backend pseudocode

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from decimal import Decimal
from typing import Literal
import time

app = FastAPI()

class EvalRequest(BaseModel):
    asset: Literal["USDY", "mETH"]
    intended_trade_usd: Decimal
    side: Literal["increase", "decrease"]

class EvalResponse(BaseModel):
    asset: str
    risk_score: float
    veto: bool
    reasons: list[str]
    approval_payload: dict

def get_pyth_eth_usd():
    # Hermes latest update + parsed price
    # Return price, confidence_ratio, publish_time, update_blob
    ...

def get_ondo_usdy_price():
    # Read Mantle Redemption Price Oracle via RPC
    # Return price_e18, updated_at
    ...
```

```
def get_dex_quotes(asset: str, notional_usd: Decimal):
    # Quote Merchant Moe + Agni
    # Return best route, quoted mid, effective slippage, depth_1pct
    ...

def build_subscores(asset, ref_price, market_price, conf_ratio, staleness_s,
depth_mult):
    reasons = []
    veto = False

    if asset == "USDY":
        market_gap = abs((market_price / ref_price) - Decimal("1"))
        M = min(float(market_gap / Decimal("0.015")), 1.0)
    else:
        discount = max(Decimal("0"), Decimal("1") - (market_price /
ref_price))
        M = min(float(discount / Decimal("0.025")), 1.0)

    O = min(float(conf_ratio / Decimal("0.0075")), 1.0)
    if staleness_s > 600:
        O = 1.0
        veto = True
        reasons.append("oracle stale")

    if depth_mult < 2:
        veto = True
        reasons.append("insufficient liquidity")
    L = 1.0 if depth_mult <= 2 else max(0.0, 1 - min(depth_mult / 5, 1))

    # Hackathon defaults; replace with empirical rolling metrics later
    Y = 0.15
    C = 0.35 if asset == "USDY" else 0.25
    S = 0.20 if asset == "USDY" else 0.45
    X = 0.10

    score = 100 * (0.25*M + 0.20*L + 0.15*Y + 0.15*C + 0.10*S + 0.10*O +
0.05*X)

    if score >= 70:
        veto = True
        reasons.append("score >= 70")
    elif score >= 50:
        reasons.append("reduce-only")

    return score, veto, reasons

@app.post("/risk/evaluate", response_model=EvalResponse)
def evaluate(req: EvalRequest):
    eth = get_pyth_eth_usd()
    dex = get_dex_quotes(req.asset, req.intended_trade_usd)
```

```
    if req.asset == "USDY":
        ref_price, updated_at = get_ondo_usdy_price()
    else:
        # For mETH use ETH as external anchor and DEX market discount as
primary signal
        ref_price, updated_at = eth["price"], eth["publish_time"]

    staleness_s = int(time.time()) - int(updated_at)
    score, veto, reasons = build_subscores(
        req.asset,
        Decimal(str(ref_price)),
        Decimal(str(dex["market_price"])),
        Decimal(str(eth["confidence_ratio"])),
        staleness_s,
        Decimal(str(dex["depth_multiple"]))
    )

    approval = {
        "asset": req.asset,
        "riskScore": round(score, 2),
        "expiresAt": int(time.time()) + 120,
        "oracleRefPriceE18": str(ref_price),
        "maxDeviationBps": 150 if req.asset == "USDY" else 250,
        # plus EIP-712 signature added by your signer service
    }

    return EvalResponse(
        asset=req.asset,
        risk_score=round(score, 2),
        veto=veto,
        reasons=reasons,
        approval_payload=approval
    )
```

The critical implementation point is that the backend should compute both a **continuous score** and **discrete vetoes**. Judges usually reward this because it demonstrates that you understand the difference between “reduce size” and “do not trade at all”. That distinction is well aligned with the documented RWA and liquidity-buffer risks in the source material. [39]

Testing and simulation

The best testing stack is a **three-layer approach**:

1. **Pure unit tests** for every scoring rule and every veto branch.
2. **Mainnet-fork integration tests** for realistic USDY/mETH behaviour.
3. **Mantle Sepolia operational tests** for signer flow, pause/unpause, Pyth update fee handling, and event indexing.

That split is necessary because the gathered official sources confirm Mantle Sepolia as the current testnet, but do not confirm live Sepolia deployments for the actual RWA assets and DEX routes your strategy cares about. [43]

Minimal test plan

Test layer	What to test	Pass criteria
Unit tests	Score normalisation, veto thresholds, expiry handling, depeg logic	100% branch coverage on veto logic
Mainnet fork	USDY oracle reads, Pyth fee flow, DEX quote comparison, executor hard checks	No trade executes if any hard guard is tripped
Sepolia smoke tests	Pause/unpause, signature verification, event emissions, storage writes	Guardian state transitions behave exactly as expected
Failure drills	Stale oracle, empty liquidity, malformed approval, chain-health flag	Transaction reverts with precise reason and emits block/veto event
Replay tests	Feed recorded quotes/oracle values through the scorer	Output classification matches expected policy bucket

Historical and simulation sources

The highest-confidence historical source already documented by Ondo is the **Redemption Price Oracle itself**, because Ondo explicitly states historical prices are stored in that contract. That makes the oracle contract a natural backtest anchor for USDY's "reference value" series. For market behaviour, use DEX pool and router events from Mantle logs via RPC or explorer APIs. If you later confirm an official subgraph for Merchant Moe or Agni, use it as a convenience layer, not as the source of truth. [44]

For external oracle history, Pyth's developer hub includes historical-data / benchmarks tooling, but this report did not verify the exact endpoint or schema from the gathered pages. In practice, that means your hackathon-safe path is:

- use Ondo oracle history for the USDY reference series,
- use your own archived DEX quotes/logs for market series,
- optionally add Pyth historical feeds once you have copied the exact benchmark endpoint from the current docs. [45]

Suggested simulation scenarios

Scenario	What you inject	Expected guardian response
USDY mild dislocation	DEX price 0.9% above Ondo oracle	Shrink size or warn
USDY hard dislocation	DEX price 1.8% away from Ondo oracle	Veto
mETH exit stress	mETH discount widens to 2.8%, depth collapses	Reduce-only or veto
Pyth stale / delayed	ETH/USD publish time too old or confidence too wide	Veto
Chain-health fault	RPC failure, quote divergence across providers, or manual chain health flag	Pause strategy

Scenario	What you inject	Expected guardian response
Privileged-role shock	Recently upgraded dependency or changed privileged role	Pause for manual review

Open questions and limitations

The following items remain unresolved from the gathered official material and should be treated as **open integration tasks**, not silent assumptions:

- **USDY on Mantle Sepolia**: UNSPECIFIED. No current Sepolia deployment was confirmed.
- **mETH on Mantle Sepolia**: UNSPECIFIED. Only the Mantle mainnet L2 token address was confirmed.
- **Merchant Moe on Mantle Sepolia**: UNSPECIFIED in the gathered sources.
- **Agni on current Mantle Sepolia**: UNSPECIFIED. Agni's SDK still exposes a generic testnet set, but it conflicts with Mantle's official current Sepolia WMNT, so it should not be treated as current truth without verification. [46]

Two oracle-specific uncertainties also remain:

- **Pyth USDY feed**: no official USDY feed was confirmed in the gathered sources.
- **Full hex for the stable-equivalent USDC/USD Pyth feed**: only an official partial snippet was surfaced in the gathered material, so the full feed ID should be copied directly from Pyth's official feed list before implementation. [31]

The safest project plan, therefore, is:

- **mainnet addresses as source of truth**,
- **mainnet fork for economic realism**,
- **Sepolia for guardian mechanics**,
- **Ondo oracle as the USDY anchor**,
- **Pyth ETH/USD as the core external oracle**,
- **strict hard vetoes on stale data, poor liquidity, and large market/reference deviations**.

That is the most defensible implementation path supported by the currently documented Mantle, Ondo, Merchant Moe, Agni, Pyth, Mantlescan/Etherscan, Alchemy, QuickNode, Gemini, and Ollama materials gathered for this report. [47]

---

[1] [2] [6] [8] [9] [23] [35] [43] [46] [47] https://docs-v2.mantle.xyz/devs/dev-guides/quick
https://docs-v2.mantle.xyz/devs/dev-guides/quick

[3] [27] https://ai.google.dev/api
https://ai.google.dev/api

[4] [7] [19] [26] [36] [39] [40] https://docs.ondo.finance/trust-and-security
https://docs.ondo.finance/trust-and-security

[5] [17] [22] [30] https://docs.pyth.network/price-feeds/core/contract-addresses/evm
https://docs.pyth.network/price-feeds/core/contract-addresses/evm

10 32 33 44 https://docs.ondo.finance/addresses
https://docs.ondo.finance/addresses

11 Mantle LSP Smart Contracts
https://github.com/mantle-lsp/contracts?utm_source=chatgpt.com

12 13 https://app.mantle.xyz/faq
https://app.mantle.xyz/faq

14 28 34 Contracts | Merchant Moe
https://docs.merchantmoe.com/resources/contracts

15 16 https://github.com/agni-protocol/agni-sdk
https://github.com/agni-protocol/agni-sdk

18 21 41 42 https://docs.pyth.network/price-feeds/core/how-pyth-works/hermes
https://docs.pyth.network/price-feeds/core/how-pyth-works/hermes

20 https://www.alchemy.com/docs/best-practices-when-using-alchemy
https://www.alchemy.com/docs/best-practices-when-using-alchemy

24 45 https://docs.pyth.network/price-feeds/core/fetch-price-updates
https://docs.pyth.network/price-feeds/core/fetch-price-updates

25 https://mantlescan.xyz/login
https://mantlescan.xyz/login

29 https://docs.ollama.com/api/authentication
https://docs.ollama.com/api/authentication

31 https://docs.pyth.network/price-feeds/core/push-feeds/evm
https://docs.pyth.network/price-feeds/core/push-feeds/evm

37 https://app.methprotocol.xyz/mantlelsp-terms
https://app.methprotocol.xyz/mantlelsp-terms

38 https://docs.mantle.xyz/network/system-information/risk-management
https://docs.mantle.xyz/network/system-information/risk-management